

Desenvolvimento *Web* III

Tecnologia em Análise e Desenvolvimento de Sistemas

Aula 9 – Acesso a banco de dados (2)

Introdução

Como visto anteriormente, utilizamos o Prisma para fazer a conexão com um banco de dados.

Nosso *site* possui componentes renderizados no cliente, que fazem requisições a funções que estão no servidor, e essas utilizam o Prisma para se comunicar com o banco de dados.

Nesta aula vamos ver mais recursos com o Prisma.

Introdução

No exemplo da aula anterior, fizemos as operações CRUD básicas para um livro, que possui identificador, título e ano de publicação, todos obrigatórios.

Recupere o projeto que está disponível no SUAP, e siga os seguintes passos:

1. Execute o comando **npm install** para fazer o *download* do código dos módulos utilizados;

Introdução

2. Inicie o Docker;
3. Suba um *container* do Docker com a imagem do Postgre utilizada:

```
docker run --name postgres -e POSTGRES_PASSWORD=senhadobanco -p 5432:5432 -v postgres:/var/lib/postgresql/data -d postgres:17
```

Introdução

4. Execute o comando **npx prisma db push** para gerar o código de acesso ao banco de dados e enviar as alterações (de esquema) para a base de dados;
 5. Execute o comando **npx prisma generate** para gerar os arquivos de acesso;
 6. Execute o servidor com o comando **npm run dev**.
- O *site* deve funcionar como na aula anterior.

Filtragem de dados

Nós utilizamos o comando **findMany** do Prisma na função **Listar** para obter todos os itens de uma tabela, sem filtrar ou ordenar o resultado.

```
export async function Listar() {  
  const livros = await prisma.livro.findMany();  
  return livros;  
}
```

Filtragem de dados

Para filtrar os dados, passamos um objeto como argumento da função **findMany**, com um objeto com um atributo **where**, e dentro deste objeto, as condições desejadas.

Filtragem de dados

Por exemplo, a função a seguir retorna apenas livros que possuem exatamente o mesmo título informado:

```
export async function PesquisarPorTitulo({ tituloAPesquisar }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: tituloAPesquisar  
    }  
  });  
  return livros;  
}
```


Filtragem de dados

No exemplo anterior, **titulo: tituloAPesquisar** realiza a comparação por igualdade entre o título cadastrado no banco de dados e o argumento recebido.

Para cada atributo, é possível fornecer um objeto com mais detalhes sobre a comparação a ser realizada.

Caso a comparação seja de igualdade, podemos usar o atributo **equals**.

Filtragem de dados

```
export async function PesquisarPorTitulo({ tituloAPesquisar }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        equals: tituloAPesquisar  
      }  
    }  
  });  
  return livros;  
}
```

Filtragem de dados

Para tipos de dados *string*, podemos usar também os seguintes atributos:

- **startsWith**, para indicar que a *string* deve iniciar com o valor informado;
- **endsWith**, para indicar que a *string* deve terminar com o valor informado;
- **contains**, para indicar que a *string* deve possuir o valor informado em qualquer parte da sua informação.

Filtragem de dados

```
export async function PesquisarPorTitulo({ tituloAPesquisar }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: tituloAPesquisar  
      }  
    }  
  });  
  return livros;  
}
```

Filtragem de dados

Crie a função anterior no arquivo “app/livros/actions.js”.

Para pesquisar livros pelo título, em “app/livros/page.js”, crie um estado para informar os dados da pesquisa a ser realizada:

```
const [pesquisaobj, setPesquisaObj] = useState({  
  titulo: ''  
});
```

Filtragem de dados

Na função **handleAtualizar**, substitua o método Listar pelo método criado:

```
try {  
    const livros = await PesquisarPorTitulo({ tituloAPesquisar:  
pesquisaobj.titulo });  
    setDados(livros);  
}
```

Não se esqueça de fazer sua importação:

```
import { PesquisarPorTitulo, Remover } from "../actions";
```

Filtragem de dados

Por fim, crie uma entrada de dados para informar qual o título de livro que deva ser buscado:

```
<div className="my-2">
  <label>
    Título
    <input type="text" className="bg-gray-100 p-2 ml-2"
value={pesquisaobj.titulo} onChange={(e) => setPesquisaObj(p => ({
...pesquisaobj, titulo: e.target.value })))} />
  </label>
</div>
```

```
use client';

import { use, useEffect, useState } from "react";
import Swal from "sweetalert2";
import { Listar, PesquisarPorTitulo, Remover } from "../actions";
import Link from "next/link";

export default function Livros() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);
  const [atualizar, setAtualizar] = useState(true);
  const [pesquisaobj, setPesquisaobj] = useState({
    titulo:
  });

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await PesquisarPorTitulo({ tituloPesquisar: pesquisaobj.titulo });
      setDados(livros);
    }
    catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: "error",
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }

    setAtualizar(false);
    setBusy(false);
  }

  const handleRemover = async (id) => {
    const resposta = confirm("Deseja realmente remover este livro?");
    if (resposta) {
      setBusy(true);
      const resultado = await Remover(id);
      if (resultado.sucesso) {
        setAtualizar(true);
        Swal.fire({
          text: "Livro removido com sucesso",
          icon: "success",
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      else {
        Swal.fire({
          text: "Erro ao remover o livro informado",
          icon: "error",
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      setBusy(false);
    }
  }

  useEffect(() => {
    if (atualizar)
      handleAtualizar();
  }, [atualizar]);

  return (
    <div>

      <p>livros disponiveis</p>

      <div className="my-2">
        <label>
          <input type="text" className="bg-gray-100 p-2 m-2" value={pesquisaobj.titulo} onChange={(e) => setPesquisaobj(p => ({ ...pesquisaobj, titulo: e.target.value })} />
        </label>
      </div>

      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" disabled={busy} onClick={() => { setAtualizar(true) }}>
        {busy ? "....." : "Atualizar"}
      </button>

      <link className="m-2 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" href="/livros/novo">Novo</link>

      <table className="w-full">
        <thead>
          <tr>
            <td>Titulo</td>
            <td>ano de Publicação</td>
          </tr>
        </thead>
        <tbody>
          {
            dados.map(p) => {
              return (
                <tr key={p.id}>
                  <td>{p.titulo}</td>
                  <td>{p.anoPublicacao}</td>
                  <td><link href="/livros/editar/${p.id}" className="text-blue-600 pl-2">editar</link></td>
                  <td><a href="#" className="text-red-600 pl-2" onClick={() => { handleRemover(p.id) }}>remover</a></td>
                </tr>
              )
            }
          }
        </tbody>
      </table>
    </div>
  )
}
```


Filtragem de dados

Para pesquisar pelo ano, crie a função em actions.js:

```
export async function PesquisarPorAno({ anoAPesquisar }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      anoPublicacao: anoAPesquisar  
    }  
  });  
  return livros;  
}
```

Filtragem de dados

A consulta anterior realiza a filtragem do número inteiro por igualdade.

Também podemos utilizar um atributo, da mesma forma que fizemos com *strings*, para utilizar mais informações na filtragem a ser realizada.

Filtragem de dados

Para tipos de dados inteiro, podemos usar também os seguintes atributos:

- **equals**, para indicar igualdade de valores;
- **lt**, para indicar que o valor deve ser menor que o informado;
- **lte**, para indicar que o valor deve ser menor ou igual ao informado;
- **gt**, para indicar que o valor deve ser maior que o informado;
- **gte**, para indicar que o valor deve ser maior ou igual ao informado;

Filtragem de dados

Em “app/livros/page.js”, atualize o objeto de pesquisa:

```
const [pesquisaobj, setPesquisaObj] = useState({  
  titulo: '',  
  anoPublicacao: ''  
});
```

Atualize a função **handleAtualizar** para realizar a pesquisa apenas por ano:

Filtragem de dados

```
try {  
    const livros = await PesquisarPorAno({ anoAPesquisar:  
parseInt(pesquisaobj.anoPublicacao) });  
    setDados(livros);  
}
```

Não esqueça de fazer sua importação:

```
import { PesquisarPorAno, Remover } from "./actions";
```

Filtragem de dados

Por fim, atualize a interface para possuir um campo onde o ano deve ser informado:

```
<div className="my-2">
  <label>
    Título
    <input type="text" className="bg-gray-100 p-2 ml-2" value={pesquisaobj.titulo}
onChange={(e) => setPesquisaObj(p => ({ ...pesquisaobj, titulo: e.target.value })))} />
  </label>

  <label className="ml-4">
    Ano
    <input type="number" className="bg-gray-100 p-2 ml-2"
value={pesquisaobj.anoPublicacao} onChange={(e) => setPesquisaObj(p => ({
...pesquisaobj, anoPublicacao: e.target.value })))} />
  </label>
</div>
```

```
'use client';

import { use, useEffect, useState } from "react";
import { Swal from "sweetalert2"; }
import { Listar, PesquisarPorAno, PesquisarPorTitulo, Remover } from "../actions";
import Link from "next/link";

export default function Livros() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);
  const [atualizar, setAtualizar] = useState(true);
  const [pesquisaobj, setPesquisaobj] = useState({
    titulo: "",
    anoPublicacao: ""
  });

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await PesquisarPorAno({ anoPesquisar: parseInt(pesquisaobj.anoPublicacao) });
      setDados(livros);
    } catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: "error",
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }

    setAtualizar(false);
    setBusy(false);
  }

  const handleRemover = async (id) => {
    const resposta = confirm("Deseja realmente remover este livro?");
    if (resposta) {
      setBusy(true);
      const resultado = await Remover(id);
      if (resultado.sucesso) {
        setAtualizar(true);
        Swal.fire({
          text: "Livro removido com sucesso",
          icon: "success",
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      } else {
        Swal.fire({
          text: "Erro ao remover o livro informado",
          icon: "error",
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      setBusy(false);
    }
  }

  useEffect(() => {
    if (atualizar)
      handleAtualizar();
  }, [atualizar]);

  return (
    <div>

      <p>Livros disponíveis</p>

      <div className="my-2">
        <label>
          <input type="text" className="bg-gray-100 p-2 ml-2" value={pesquisaobj.titulo} onChange={(e) => setPesquisaobj(p => ({ ...pesquisaobj, titulo: e.target.value }))) />
        </label>

        <label className="ml-4">
          <input type="number" className="bg-gray-300 p-2 ml-2" value={pesquisaobj.anoPublicacao} onChange={(e) => setPesquisaobj(p => ({ ...pesquisaobj, anoPublicacao: e.target.value }))) />
        </label>
      </div>

      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 disabled:busy" onClick={() => { setAtualizar(true) }}>
        Busy ? "....." : "Atualizar"
      </button>

      <link className="ml-2 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 href="/livros/novo">Novo</link>

      <table className="w-full">
        <thead>
          <tr>
            <td>Titulo</td>
            <td>Ano de Publicação</td>
          </tr>
        </thead>
        <tbody>
          {
            dados.map(p => {
              return (
                <tr key={p.id}>
                  <td>{p.titulo}</td>
                  <td>{p.anoPublicacao}</td>
                  <td><link href="/livros/editar/${p.id}" className="text-blue-600 pl-2">Editar</link></td>
                  <td><a href="#" className="text-red-600 pl-2" onClick={() => { handleRemover(p.id) }}>Remover</a></td>
                </tr>
              )
            })
          }
        </tbody>
      </table>

    </div>
  )
}
```

Filtragem de dados

O código anterior, no entanto, não funciona caso o campo não seja preenchido.

Para isso, vamos alterar o código para o seguinte:

```
export async function PesquisarPorAno({ anoAPesquisar }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      ...(!isNaN(Number(anoAPesquisar)) && {  
        anoPublicacao: anoAPesquisar  
      })  
    }  
  });  
  return livros;  
}
```


Filtragem de dados

A função **Number** transforma um valor em número (ou NaN, caso a conversão apresente erros).

A função **isNaN** retorna verdadeiro se um valor não é um número, ou falso caso contrário.

Assim, caso o valor informado não seja um número, o filtro não é realizado.

Caso o valor seja um número, acrescenta-se no objeto de comparação o valor a ser pesquisado.

Filtragem de dados

Para evitar fazer uma função de pesquisa para cada campo, vamos juntar as duas funções em uma só.

Remova as funções de pesquisa por ano e pesquisa por título.

Crie uma nova função que pesquisa por ambos:

Filtragem de dados

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(!isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    }  
  });  
  return livros;  
}
```

Filtragem de dados

Em “app/livros/page.js”, atualize a função **handleAtualizar** para chamar a nova função:

```
try {  
    const livros = await Pesquisar({ ...pesquisaobj, ano:  
parseInt(pesquisaobj.anoPublicacao) });  
    setDados(livros);  
}
```

A pesquisa deverá funcionar, fazendo um **AND** entre o título e o ano de publicação informado.

```
'use client';

import { useEffect, useState } from 'react';
import Swal from 'sweetalert2';
import { Pesquisar, Remover } from './actions';
import Link from 'next/link';

export default function Livros() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);
  const [atualizar, setAtualizar] = useState(true);
  const [pesquisaobj, setPesquisaobj] = useState({
    titulo: '',
    anoPublicacao: ''
  });

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const livros = await Pesquisar([ ...pesquisaobj, ano: parseInt(pesquisaobj.anoPublicacao) ]);
      setDados(livros);
    }
    catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: "error",
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
  }

  setAtualizar(false);
  setBusy(false);
}

const handleRemover = async (id) => {
  const resposta = confirm("Deseja realmente remover este livro?");
  if (resposta) {
    setBusy(true);
    const resultado = await Remover(id);
    if (resultado.sucesso) {
      setAtualizar(true);
      Swal.fire({
        text: "Livro removido com sucesso",
        icon: "success",
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
    else {
      Swal.fire({
        text: "Erro ao remover o livro informado",
        icon: "error",
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
    setBusy(false);
  }
}

useEffect(() => {
  if (atualizar)
    handleAtualizar();
}, [atualizar]);

return (
  <div>

    <p>livros disponiveis</p>

    <div className="my-2">
      <label>
        <input type="text" className="bg-gray-100 p-2 ml-2" value={pesquisaobj.titulo} onChange={(e) => setPesquisaobj(p => ({ ...pesquisaobj, titulo: e.target.value }))) />
      </label>
      <label className="ml-4">
        <input type="number" className="bg-gray-300 p-2 ml-2" value={pesquisaobj.anoPublicacao} onChange={(e) => setPesquisaobj(p => ({ ...pesquisaobj, anoPublicacao: e.target.value }))) />
      </label>
    </div>

    <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 disabled:busy" onClick={() => { setAtualizar(true) }}>
      Busy ? "....." : "Atualizar"
    </button>
    <link className="ml-2 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 href="/livros/novo">Novo</link>

    <table className="w-full">
      <thead>
        <tr>
          <td>Titulo</td>
          <td>Ano de Publicação</td>
        </tr>
      </thead>
      <tbody>
        {
          dados.map(p) => {
            return (
              <tr key={p.id}>
                <td>{p.titulo}</td>
                <td>{p.anoPublicacao}</td>
                <td><link href="/livros/editar/${p.id}" className="text-blue-600 pl-2">Editar</link></td>
                <td><a href="#" className="text-red-600 pl-2" onClick={() => { handleRemover(p.id) }}>Remover</a></td>
              </tr>
            )
          }
        }
      </tbody>
    </table>

  </div>
)
```

Filtragem de dados

Caso queira fazer um OR entre valores, deve-se informar um vetor de objetos utilizando o operador OR:

```
where: {  
  OR: [  
    { /* condição 1 */ },  
    { /* condição 2 */ },  
    { /* condição n */ }  
  ]  
}
```

Filtragem de dados

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      OR: [  
        {  
          titulo: {  
            contains: titulo  
          }  
        },  
        {  
          ...(!isNaN(Number(ano)) && {  
            anoPublicacao: ano  
          })  
        }  
      ]  
    }  
  });  
  return livros;  
}
```

Filtragem de dados

Também é possível verificar por diferença. Por exemplo, para trazer todos os livros que **não** foram publicados no ano informado:

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      anoPublicacao: {  
        not: ano  
      }  
    }  
  });  
  return livros;  
}
```


Ordenação de dados

Por padrão, os dados retornados do banco de dados são ordenados pelo ordem crescente de criação.

Para ordenar os resultados por uma coluna, utilize no objeto da função **findMany** um objeto **orderBy**, que deve possuir como atributo o campo a ser ordenado, com valor “asc” ou “desc” para ordenar de forma ascendente ou descendente.

Ordenação de dados

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(!isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: {  
      titulo: "asc"  
    }  
  });  
  return livros;  
}
```

Ordenação de dados

Para fazer a ordenação utilizando mais de uma coluna, o valor de **orderBy** deve ser um vetor de objetos, em que cada objeto possui como atributo o nome da coluna a ser ordenada e como valor o tipo de ordenação.

A ordenação é feita do objeto no menor índice do vetor para o objeto no maior índice do vetor.

Ordenação de dados

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: [  
      {  
        titulo: "asc"  
      },  
      {  
        anoPublicacao: "desc"  
      }  
    ]  
  });  
  return livros;  
}
```

Seleção de colunas

Por padrão, todas as colunas da tabela são retornadas em uma consulta.

Caso queira trazer apenas algumas delas, utilize no objeto argumento da função **findMany** o objeto **select**, que deve possuir como valor um objeto contendo atributos com os nomes das colunas que queira retornar, contendo o valor **true**.

Para não recuperar um valor de uma coluna, basta omitir ela do objeto **select**, ou então, utilizar seu valor como **false**.

Seleção de colunas

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(!isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: {  
      titulo: "asc"  
    },  
    select: {  
      id: true,  
      titulo: true  
    }  
  });  
  return livros;  
}
```

Seleção de colunas

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: {  
      titulo: "asc"  
    },  
    select: {  
      id: true,  
      titulo: true,  
      anoPublicacao: false  
    }  
  });  
  return livros;  
}
```

Seleção de colunas

Em casos em que você deseja recuperar várias colunas e omitir poucas, ao invés de utilizar o objeto **select**, podemos utilizar o objeto **omit**, que deve possuir como atributo o nome da coluna a se omitir, e como valor o valor **true**.

Caso seja utilizado o valor **false**, a coluna não será omitida da pesquisa.

Seleção de colunas

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(!isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: {  
      titulo: "asc"  
    },  
    omit: {  
      anoPublicacao: true  
    }  
  });  
  return livros;  
}
```

Seleção de colunas

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(!isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: {  
      titulo: "asc"  
    },  
    omit: {  
      id: false,  
      titulo: false,  
      anoPublicacao: true  
    }  
  });  
  return livros;  
}
```

Paginação

Para definir a quantidade máxima de itens retornados da consulta, utilize o atributo **take** dentro do objeto passado como argumento da função **findMany**.

Paginação

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(!isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: {  
      titulo: "asc"  
    },  
    take: 4  
  });  
  return livros;  
}
```

Paginação

Para definir a quantidade de itens ignorados da consulta, utilize o atributo **skip** dentro do objeto passado como argumento da função **findMany**.

Paginação

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(!isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: {  
      titulo: "asc"  
    },  
    take: 4,  
    skip: 2  
  });  
  return livros;  
}
```

Consultas SQL

Caso não queira usar a sintaxe do Prisma para criar uma consulta, utilize o atributo `queryRaw`:

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.$queryRaw`  
    select *  
    from "Livro"  
    where titulo like ${'%' + titulo + '%'} and "anoPublicacao"=${ano} `;  
  return livros;  
}
```

Essa pesquisa não permite ano NULL!

Editora

Antes de trabalhar com relacionamentos, vamos criar uma nova tabela, responsável pelo cadastro de editoras, das quais vamos armazenar os seguintes campos:

- Identificador
- Nome (máximo de 100 caracteres)
- Sigla (única)
- Cidade (opcional)

Editora

Atualize o arquivo “schema.prisma” com a seguinte *model*:

```
model Editora {  
  id Int @id @default (autoincrement())  
  nome String @db.VarChar(100)  
  sigla String @unique  
  cidade String?  
}
```

Editora

Utilize o comando **npx prisma db push** para enviar as alterações para a base de dados.

Utilize o comando **npx prisma generate** para atualizar os arquivos de conexão.

Editora

Em “app/page.js”, crie um Link para a página de editoras:

```
import Link from "next/link";

export default function Home() {
  return (
    <>
      <Link href="/livros">Livros</Link>
      <Link href="/editora">Editoras</Link>
    </>
  )
}
```

Editora

Crie as pastas:

- “app/editora”
- “app/editora/novo”
- “app/editora/[id]”

Editora

Em “app/editora”, crie o arquivo actions.js, com as funções que serão executadas no servidor, responsáveis pelas operações CRUD com a *model* de editoras.

```
'use server'

import { prisma } from "../../lib/prisma";

export async function Listar() {
  const resultado = await prisma.editora.findMany();
  return resultado;
}

export async function Salvar(editora) {
  const resultado = await prisma.editora.create({
    data: editora
  });

  return resultado;
}

export async function Obter(id) {
  const resultado = prisma.editora.findUnique({
    where: {
      id: id
    }
  });

  return resultado;
}

export async function Atualizar(editora) {
  const resultado = prisma.editora.update({
    where: {
      id: editora.id
    },
    data: { ...editora, id: undefined }
  });

  return resultado;
}

export async function Remover(id) {
  const resultado = prisma.editora.delete({
    where: {
      id: id
    }
  });
  return resultado;
}
```

Editora

Em “app/editora/”, crie o arquivo `page.js`, contendo o código para exibição e remoção das editoras.

```
'use client';

import { useEffect, useState } from "react";
import Swal from 'sweetalert2';
import { Listar, Remover } from "../actions";
import Link from "next/link";

export default function Editoras() {

  const [busy, setBusy] = useState(false);
  const [dados, setDados] = useState([]);
  const [atualizar, setAtualizar] = useState(true);

  const handleAtualizar = async () => {
    setBusy(true);

    try {
      const resultado = await Listar();
      setDados(resultado);
    }
    catch {
      Swal.fire({
        text: "Erro ao buscar os dados",
        icon: "error",
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }

    setAtualizar(false);
    setBusy(false);
  }

  const handleRemover = async (id) => {
    const resposta = confirm("Deseja realmente remover este editora?");
    if (resposta) {
      setBusy(true);
      const resultado = await Remover(id);
      if (resultado) {
        setAtualizar(true);
        Swal.fire({
          text: "editora removida com sucesso",
          icon: 'success',
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      else {
        Swal.fire({
          text: "Erro ao remover a editora informada",
          icon: 'error',
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      setBusy(p => false);
    }
  }

  useEffect(() => {
    if (atualizar) {
      handleAtualizar();
    }
  }, [atualizar]);

  return (
    <div>

      <button className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 disabled={busy} onClick={() => { setAtualizar(true) }}>
        Busy ? "....." : "Atualizar"
      </button>

      <Link className="ml-4 bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200" href="/editora/novo">Novae/Link</Link>

      <table className="w-full">
        <thead>
          <tr>
            <td>Nome</td>
            <td className="px-4">Sigla</td>
            <td>Cidade</td>
            <td className="px-4">Editar</td>
            <td>Remover</td>
          </tr>
        </thead>
        <tbody>
          {
            dados.map((p) => {
              return (
                <tr key={p.id}>
                  <td>{p.nome}</td>
                  <td className="px-4">{p.sigla}</td>
                  <td>{p.cidade}</td>
                  <td><Link href="/editora/editar/${p.id}" className="text-blue-600 px-4">Editar</Link></td>
                  <td><span className="text-red-600 cursor-pointer" onClick={() => { handleRemover(p.id) }}>Remover</span></td>
                </tr>
              )
            })
          }
        </tbody>
      </table>

    </div>
  )
}
```


Editora

Em “app/editora”, crie o arquivo schema.js, contendo o objeto de validação para editora.

```
import * as yup from "yup"

export const EditoraSchema = yup.object({
  nome: yup.string()
    .required('O nome é obrigatório')
    .max(100, 'O tamanho máximo do nome da editora é de 100
caracteres'),
  sigla: yup.string()
    .required('A sigla é obrigatória'),
  cidade: yup.string()
}).required();
```

Editora

Em “app/editora/novo”, crie o arquivo `page.js`, contendo o código para cadastro de uma nova editora.

```
'use client'

import { useState } from "react"

import Swal from 'sweetalert2';
import { useRouter } from "next/navigation";
import { Salvar } from "../actions";
import { useForm } from "react-hook-form";
import { EditoraSchema } from "../schema";
import { yupResolver } from "@hookform/resolvers/yup"

export default function Novo() {

  const router = useRouter();

  const [busy, setBusy] = useState(false);

  const { register, handleSubmit, formState: { errors } } = useForm({
    defaultValues: {
      nome: '',
      sigla: '',
      cidade: ''
    },
    resolver: yupResolver(EditoraSchema),
  });

  const onSubmit = async (data) => {
    setBusy(true);

    try {
      const resultado = await Salvar(data);
      if (resultado) {
        Swal.fire({
          text: 'Editora salva com sucesso',
          icon: 'success',
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
        router.push("/editora");
      }
    } catch {
      Swal.fire({
        text: 'Erro ao salvar a editora informada',
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }

    setBusy(p => false);
  }

  return (
    <div className="mt-2 ml-2">
      <form onSubmit={handleSubmit(onSubmit)} className="flex flex-row">
        <div className="flex flex-col space-y-1 text-right">
          <label>
            Nome
            <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("nome")} />
            <p className="text-sm text-red-600">{errors?.nome?.message}</p>
          </label>
          <label>
            Sigla
            <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("sigla")} />
            <p className="text-sm text-red-600">{errors?.sigla?.message}</p>
          </label>
          <label>
            Cidade
            <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("cidade")} />
            <p className="text-sm text-red-600">{errors?.cidade?.message}</p>
          </label>
          <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 ml-2" disabled={busy}>
            {busy ? " ..... " : "Salvar"}
          </button>
        </div>
      </form>
    </div>
  )
}
```

Editora

Em “app/editora/editar/[id]”, crie o arquivo page.js, contendo o código para edição de uma editora.

```
'use client'

import { use, useEffect, useState } from "react"

import Swal from "swalalert2";
import { useRouter } from "next/navigation";
import { useForm } from "react-hook-form";
import { yupResolver } from "hookform/resolvers/yup"
import { equalizar, obter } from ".././././actions";
import { EditorSchema } from ".././././schema";

export default function Editor({ params }) {

  const router = useRouter();
  const [busy, setBusy] = useState(false);
  const { id } = use(params);

  const { register, handleSubmit, reset, formState: { errors } } = useForm({
    defaultValues: {
      nome: '',
      sigla: '',
      cidade: ''
    },
    resolver: yupResolver(EditorSchema),
  });

  const onSubmit = async (data) => {
    setBusy(true);
    data.id = parseInt(id);

    try {
      const resultado = await Atualizar(data);
      if (resultado) {
        Swal.fire({
          text: 'Editora salva com sucesso',
          icon: 'success',
          timer: 3000,
          toast: true,
          position: 'top-right',
          showConfirmButton: false
        })
        router.push("/editora");
      }
    } catch {
      Swal.fire({
        text: 'Erro ao salvar a editora informada',
        icon: 'error',
        timer: 3000,
        toast: true,
        position: 'top-right',
        showConfirmButton: false
      })
    }

    setBusy(p => false);
  }

  const handleCarregar = async () => {
    let atual = null;
    try {
      setBusy(true);
      const idInt = parseInt(id);
      atual = await obter(idInt);
    } catch {
      setBusy(false);
      router.replace("/editoras");
    }

    setBusy(false);
    if (atual) {
      reset({
        nome: atual.nome,
        sigla: atual.sigla,
        cidade: atual.cidade
      })
    }
    else
      router.replace("/editoras");
  }

  useEffect(() => {
    if (!id) {
      handleCarregar();
    }
  }, [id]);

  return (
    <div className="m-2 ml-2">
      <form onSubmit={handleSubmit(onSubmit)} className=" flex flex-row">
        <div className="flex flex-col space-y-1 text-right">
          <label>
            Nome
            <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("nome")} />
          </label>
          <label>
            Sigla
            <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("sigla")} />
          </label>
          <label>
            Cidade
            <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("cidade")} />
          </label>
          <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 ml-2 disabled={busy}>
            {busy ? "....." : "Salvar"}
          </button>
        </div>
      </form>
    </div>
  )
}
```

Relacionamentos

Com as *models* de editora e livro criadas, podemos criar um relacionamento entre elas.

Este relacionamento é um relacionamento de 1 para muitos, ou seja, 1 livro pode ser publicado por uma editora, mas uma editora pode publicar N livros.

Relacionamentos

Na *model* de livro, devemos adicionar uma chave estrangeira à *model* de editora, indicando a chave primária desta tabela.

Para fazer o relacionamento, também colocamos nesta *model* um objeto de Editora.

Em editora, adicionamos um vetor de Livro.

Relacionamentos

O arquivo schema.prisma ficará com as *models* assim:

```
model Livro {  
  id          Int @id @default (autoincrement())  
  titulo      String  
  anoPublicacao Int  
  editoraId    Int  
  editora      Editora @relation(fields: [editoraId], references: [id])  
}
```

```
model Editora {  
  id      Int @id @default (autoincrement())  
  nome    String @db.VarChar(100)  
  sigla   String @unique  
  cidade  String ?  
  livros  Livro[]  
}
```


Relacionamentos

Se utilizarmos o comando **npx prisma db push** para enviarmos as alterações para o banco, veremos um erro, pois como já há dados cadastrados na tabela de livro, o campo `editorald` não pode ser nulo.

Para resolver isso:

- Podemos definir um valor padrão para o campo `editorald`;
- Podemos marcar `editorald` como possível de ser nulo;
- Apagamos todos os dados.

Relacionamentos

O arquivo schema.prisma ficará assim:

```
generator client {
  provider = "prisma-client"
  output = "../generated/prisma"
}

datasource db {
  provider = "postgresql"
}

model Livro {
  id          Int @id @default (autoincrement())
  titulo      String
  anoPublicacao Int
  editoraId   Int ?
  editora     Editora ? @relation(fields: [editoraId], references: [id])
}

model Editora {
  id      Int @id @default (autoincrement())
  nome    String @db.VarChar(100)
  sigla   String @unique
  cidade  String ?
  livros  Livro[]
}
```

Relacionamentos

Execute os comandos:

- **npx prisma db push**
- **npx prisma generate**

Relacionamentos

Precisamos agora alterar as telas de cadastro e atualização de livro, adicionando um campo **select** que deve vir preenchido com as editoras cadastradas.

Em “app/livros/novo/page.js”, crie um estado para armazenar as editoras cadastradas:

```
const [editoras, setEditoras] = useState([]);
```

Relacionamentos

Atualize o schema de livro para que aceite o campo da editora:

```
import * as yup from "yup"

export const LivroSchema = yup.object({
  titulo: yup.string()
    .min(2, 'O título deve possuir, no mínimo, 2 caracteres')
    .max(100, 'O título deve possuir, no máximo, 100 caracteres')
    .required('O título é obrigatório'),
  anoPublicacao: yup.number()
    .integer()
    .min(1500, 'O valor mínimo de ano de publicação é 1500')
    .max(2026, 'O valor máximo de ano de publicação é 1500')
    .required('O ano de publicação é obrigatório')
    .TypeError('O ano de publicação é obrigatório'),
  editoraId: yup.string()
}).required();
```

Relacionamentos

Atualize o objeto inicial do Hook do React Hook Forms:

```
const { register, handleSubmit, formState:
{ errors } } = useForm({
  defaultValues: {
    titulo: '',
    anoPublicacao: '',
    editoraId: ''
  },
  resolver: yupResolver(LivrosSchema),
});
```

Relacionamentos

Crie uma função para carregar as editoras, que faz uma chamada para a função Listar do actions de editora:

Relacionamentos

```
const carregarEditoras = async () => {  
  const resultado = await Listar();  
  setEditoras(resultado);  
}
```

Não esqueça de importar corretamente a função Listar:

```
import { Listar } from "@app/editora/actions";
```


Relacionamentos

Crie um efeito que, após a montagem inicial da página, retorne as editoras cadastradas:

```
useEffect(() => {  
    carregarEditoras();  
}, []);
```

```
import { useEffect, useState } from "react"
```

Relacionamentos

Dentro do formulário, crie um **select** que será preenchido com os valores das editoras recuperadas:

```
<label>
  Editora
  <select className="bg-neutral-50 border m1-2 px-2 py-2"
{...register("editoraId")}>
    <option key={0} value=''>[Nenhuma]</option>
    {
      editoras?.map(p => (
        <option key={p.id} value={p.id} label={p.nome} />
      ))
    }
  </select>
</label>
```

```
'use client'

import { useEffect, useState } from "react"
import { useForm } from "react-hook-form"
import { yupResolver } from "@hookform/resolvers/yup"
import { LivroSchema } from "../schema";
import { salvar } from "../actions";
import Swal from 'sweetalert2';
import { useRouter } from "next/navigation";
import { Listar } from "@app/editora/actions";

export default function Novo() {

  const [busy, setBusy] = useState(false);
  const router = useRouter();

  const [editoras, setEditoras] = useState([]);

  const { register, handleSubmit, formState: { errors } } = useForm({
    defaultValues: {
      titulo: '',
      anoPublicacao: '',
      editoraid: ''
    },
    resolver: yupResolver(LivroSchema),
  });

  const onSubmit = async (data) => {
    setBusy(true);
    const resultado = await salvar(data);
    setBusy(false);

    if (resultado.sucesso) {
      if (resultado.mensagem) {
        Swal.fire({
          text: resultado.mensagem,
          icon: 'success',
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      router.push("/livros");
    }
    else {
      let mensagem = "Erro ao buscar os dados";
      if (resultado.mensagem)
        mensagem = resultado.mensagem;

      Swal.fire({
        text: mensagem,
        icon: 'error',
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
  }

  const carregarEditoras = async () => {
    const resultado = await Listar();
    setEditoras(resultado);
  }

  useEffect(() => {
    carregarEditoras();
  }, []);

  return (
    <div className="mt-2 ml-2">
      <form className="flex flex-row gap-8" onSubmit={handleSubmit(onSubmit)}>
        <label>
          Titulo
          <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("titulo")} />
          <p className="text-sm text-red-600">{errors?.titulo?.message}</p>
        </label>
        <label>
          Ano de publicação
          <input type="number" className="bg-neutral-50 border ml-2 px-2 py-2" {...register("anoPublicacao")} />
          <p className="text-sm text-red-600">{errors?.anoPublicacao?.message}</p>
        </label>
        <label>
          Editora
          <select className="bg-neutral-50 border ml-2 px-2 py-2" {...register("editoraid")}>
            <option key={0} value="">[Nenhuma]</option>
            {
              editoras?.map(p => (
                <option key={p.id} value={p.id} label={p.nome} />
              ))
            }
          </select>
        </label>
        <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 ml-2" disabled={busy}>
          {busy ? "....." : "salvar"}
        </button>
      </form>
    </div>
  )
}
```

Relacionamentos

Atualize a função Salvar em “app/livros/actions.js” para armazenar corretamente o valor da editora escolhida:

```
export async function salvar(livro) {  
  if (livro.editoraId === '')  
    livro.editoraId = null;  
  else  
    livro.editoraId = parseInt(livro.editoraId);  
  
  try {  
    await validar(livro);  
  }  
  catch (e) {  
    return { sucesso: false, mensagem: e };  
  }  
  
  let resultado = null;  
  try {  
    resultado = await prisma.livro.create({  
      data: livro  
    });  
  }  
  catch {  
    return { sucesso: false, mensagem: 'Erro na inserção' };  
  }  
  
  return { sucesso: true, dados: resultado, mensagem: 'Inserção realizada com sucesso' };  
}
```

Relacionamentos

Atualize a função Validar em “app/livros/actions.js” para verificar se a editora escolhida realmente existe:

```
async function validar(livro) {  
  livro.titulo = livro.titulo?.trim();  
  
  if (!livro.titulo || livro.titulo.length < 2 || livro.titulo.length > 100)  
    throw new Error('Título inválido');  
  
  if (!livro.anoPublicacao || livro.anoPublicacao < 1500 || livro.anoPublicacao > 2026)  
    throw new Error('Ano de publicação inválido');  
  
  if (livro.editoraId) {  
    const editora = await prisma.editora.findUnique({  
      where: {  
        id: livro.editoraId  
      }  
    });  
  
    if (!editora)  
      throw new Error('Editora inválida');  
  }  
}
```

Relacionamentos

Uma vez salvo o livro, na página de pesquisa, podemos ver que o objeto retornado possui apenas o id da editora escolhida (ou nulo, caso nenhuma editora tenha sido selecionada).

Para recuperar os dados da editora juntamente com os dados do livro, utilizamos o objeto **include** na função Pesquisar, em “app/livros/actions.js”:

Relacionamentos

```
export async function Pesquisar({ titulo, ano }) {  
  const livros = await prisma.livro.findMany({  
    where: {  
      titulo: {  
        contains: titulo  
      },  
      ...(!isNaN(Number(ano)) && {  
        anoPublicacao: ano  
      })  
    },  
    orderBy: {  
      titulo: "asc"  
    },  
    include: {  
      editora: true  
    }  
  });  
  return livros;  
}
```

Relacionamentos

Na página de pesquisa de livros, basta acessarmos o atributo nome do objeto editora do livro desejado, caso a editora seja diferente de nulo.

Relacionamentos

```
<table className="w-full">
  <thead>
    <tr>
      <td>Título</td>
      <td>Ano de Publicação</td>
      <td>Editora</td>
    </tr>
  </thead>
  <tbody>
    {
      dados.map((p) => {
        return (
          <tr key={p.id}>
            <td>{p.titulo}</td>
            <td>{p.anoPublicacao}</td>
            <td>{p.editora?.nome}</td>
            <td><Link href={`/livros/editar/${p.id}`} className="text-blue-600 pl-2">Editar</Link></td>
            <td><a href="#" className="text-red-600 pl-2" onClick={() => { handleRemover(p.id)
          }}>Remover</a></td>
        </tr>
      )
    }
  </tbody>
</table>
```

Relacionamentos

Para editar um livro, como feito na criação, devemos trazer todas as editoras e preencher um **select** com elas.

Essa função deve executar antes da função que busca os dados do livro.

Após recuperar os dados do livro, o valor deste campo, no estado **dados**, deve conter o valor da chave estrangeira correspondente (ou "", caso o valor seja nulo).

```
'use client'

import { use, useEffect, useState } from "react"
import { useForm } from "react-hook-form"
import { yupResolver } from "formik/resolvers/yup"
import { LivrosSchema } from "../schemas"
import { Atualizar, Obter } from "../actions"
import Swal from "sweetalert2"
import { editoras } from "next/navigation"
import { Listar } from "@app/editors/actions"

export default function Novo({ params }) {
  const [busy, setBusy] = useState(false);
  const router = useRouter();
  const { id } = useParams();
  const [editoras, setEditoras] = useState([]);

  const { register, handleSubmit, reset, formState: { errors } } = useForm({
    defaultValues: {
      titulo: "",
      anoPublicacao: "",
      editoraId: ""
    },
    resolver: yupResolver(LivrosSchema),
  });

  const onSubmit = async (data) => {
    setBusy(true);
    data.id = parseInt(id);
    const resultado = await Atualizar(data);
    setBusy(false);

    if (resultado.sucesso) {
      if (resultado.mensagem) {
        Swal.fire({
          text: resultado.mensagem,
          icon: "success",
          timer: 3000,
          toast: true,
          position: "top-right",
          showConfirmButton: false
        })
      }
      router.push("/livros");
    }
    else {
      let mensagem = "Erro ao buscar os dados";
      if (resultado.mensagem)
        mensagem = resultado.mensagem;

      Swal.fire({
        text: mensagem,
        icon: "error",
        timer: 3000,
        toast: true,
        position: "top-right",
        showConfirmButton: false
      })
    }
  }

  const handleCarregar = async () => {
    let atual = null;
    try {
      setBusy(true);
      const idInt = parseInt(id);
      await carregarEditoras();
      atual = await Obter(idInt);
    }
    catch {
      setBusy(false);
      router.replace("/livros");
    }
    setBusy(false);
    if (atual) {
      reset({
        titulo: atual.titulo,
        anoPublicacao: atual.anoPublicacao,
        editoraId: atual.editoraId ? atual.editoraId : ""
      })
    }
    else
      router.replace("/livros");
  }

  const carregarEditoras = async () => {
    const resultado = await listar();
    setEditoras(resultado);
  }

  useEffect(() => {
    if (id) {
      handleCarregar();
    }
  }, [id]);

  return (
    <div className="m-2 ml-2">
      <form className="flex flex-row gap-8" onSubmit={handleSubmit(onSubmit)}>
        <label>
          Titulo
          <input type="text" className="bg-neutral-50 border mx-2 px-2 py-2" {...register("titulo")} />
          <p className="text-sm text-red-600">{errors?.titulo?.message}</p>
        </label>
        <label>
          Ano de publicação
          <input type="number" className="bg-neutral-50 border ml-2 px-2 py-2" {...register("anoPublicacao")} />
          <p className="text-sm text-red-600">{errors?.anoPublicacao?.message}</p>
        </label>
        <label>
          Editora
          <select className="bg-neutral-50 border ml-2 px-2 py-2" {...register("editoraId")}>
            <option key={0} value="">[nenhuma]</option>
            {
              editoras?.map(p => (
                <option key={p.id} value={p.id} label={p.nome} />
              ))
            }
          </select>
        </label>
        <button type="submit" className="bg-blue-600 text-white px-4 py-2 rounded hover:bg-blue-700 active:bg-blue-800 disabled:bg-blue-200 ml-2 disabled={busy}>
          {busy ? "....." : "Salvar"}
        </button>
      </form>
    </div>
  )
}
```

Relacionamentos

Para atualizar o valor da editora, atualize a função **Atualizar** em “app/livros/actions.js”:

```
export async function Atualizar(livro) {
  if (livro.editoraId === '')
    livro.editoraId = null;
  else
    livro.editoraId = parseInt(livro.editoraId);

  try {
    await validar(livro);
  }
  catch (e) {
    return { sucesso: false, mensagem: e };
  }

  let resultado = null;
  try {
    resultado = await prisma.livro.update({
      where: {
        id: livro.id
      },
      data: { ...livro, id: undefined }
    });
  }
  catch {
    return { sucesso: false, mensagem: 'Erro na atualização' };
  }

  return { sucesso: true, dados: resultado, mensagem: 'Atualização realizada com sucesso' };
}
```